

CXL-ECC: an Efficient LRC-based on-CXL-Memory-eXpander-Controller ECC to Enhance Reliability and Performance of DRAM Error Correction

Yixuan Liu[†], Yunfei Gu[†], Junhao Dai[†], Xinyuan Wu[†], Chentao Wu^{†*}, Xinfei Guo[‡], Jieru Zhao[†], Jie Li[†], Minyi Guo[†],

[†]Shanghai Jiao Tong University, Shanghai, China

[‡]University of Michigan-Shanghai Jiao Tong University Joint Institute, Shanghai, China

*Corresponding Authors: wuct@cs.sjtu.edu.cn

Abstract—Compute eXpress Link (CXL) offers an effective interface for connecting CPUs with external computing and memory devices. CXL Memory eXpander Controller (CXL-MXC) is gaining attention for its ability to boost memory capacity and bandwidth more efficiently than traditional DDR DIMMs. Despite extensive research on MXC performance and adaptation, DRAM reliability in CXL architecture remains underexplored. Traditional fault tolerance mechanisms like replica or RAID-based systems would significantly increase bandwidth overhead in the CXL fabric, adversely affecting system performance. To address this, we propose the on-CXL-Memory-Expander-Controller ECC (CXL-ECC), by using Locally Recoverable Codes (LRC) as the Inter-Channel-ECC (IC-ECC) and offloading its process to the expander, we eliminate extra memory access requests in the CXL fabric. Consequently, we conduct several experiments to demonstrate that our approach enhances DRAM reliability by more than 10^9 , compared to state-of-the-art ECC methods. Relative to RAID-enabled CXL switch, it reduces additional bandwidth overhead from 63.5% to 3.4% and improves system performance by 12%.

Index Terms—DRAM Reliability, ECC, Compute eXpress Link

I. INTRODUCTION

Memory failures are the primary factor affecting server reliability, with over 37% of hardware faults caused by memory errors [1]. To mitigate memory errors, Error Correcting Code (ECC) [2–4] is commonly implemented in the traditional DDR memory devices. Specifically, Rank-Level ECC (R-ECC) and On-Die ECC (OD-ECC) [5] encodes data and stores error correction bits in parity DRAM chips and redundancy on DRAM chips. However, merely increasing redundancy within a DRAM device to improve reliability is insufficient. Therefore, researchers have proposed various solutions, such as Memory Mirror [6], RAIM-3 [7], and RAIM-5 [8], which implement Channel-Level Fault Tolerance (Channelkill) by using replicas or RAID [9] across memory channels. Given that these methods are integrated within memory controllers of computing devices, we categorize them as Compute Managed Type (C-Type).

In recent years, Compute eXpress Link (CXL) [10] has been proposed as an emerging open industry standard interconnect technology, inherits high-bandwidth characteristics of the PCIe protocol and provides additional cache and memory semantics, significantly reducing interface latency. This makes CXL an ideal choice for connecting CPUs and heterogeneous computing and memory devices, and it is applied in the latest storage devices, including CXL Dynamic Random Access Memory (DRAM), Solid State Drive (SSD) or Hybrid [11], and CXL Persistent Memory (PM) [12]. Extensive research has addressed

TABLE I: Comparison of Channelkill technology applied in CXL architectures

	RAIM-5	Switch-RAID	CXL-ECC
HW Modification	LLC and IMC	CXL Switch	CXL MXC
Bandwidth overhead	Medium	High	None ☺
Compatibility	Low	High	High ☺
System Overhead	Low	High	Low ☺
IC-ECC Code	XOR	XOR	LRC ☺
DRAM Reliability	Low	Low	High ☺

CXL memory prototype devices performance, focusing on operating system adaptation and memory-intensive application optimization [13–15].

CXL may bring new opportunities for memory reliability improvement. To facilitate Channelkill implementation within the emerging interconnect, some researchers have suggested Switch Managed Type (S-Type) mechanism, where the switch maintains RAID management [16]. However, when applied to the CXL architecture, both C-Type and S-Type still have two major drawbacks: ❶ Parity update introduces a substantial addition of memory requests across the CXL fabric, resulting in significant degradation of system performance. ❷ XOR-based Inter-Channel-ECC (IC-ECC) exhibits poor capability of error correction in addressing multiple random faults.

To enhance high memory reliability and optimal system performance simultaneously, we propose an efficient on-CXL-Memory-eXpander-Controller ECC (CXL-ECC) approach in this paper. ❶ This method confines the parity update related memory requests within the MXC, thus reducing external memory bandwidth usage and improving system efficiency. ❷ It innovatively implements Locally Recoverable Codes (LRC) [17] as the IC-ECC within the CXL MXC for fault tolerance in extensive fine-grained faults while maintaining Channelkill. Table I compares RAIM-5 (SOTA Type-C), Switch-RAID (SOTA Type-S) and CXL-ECC across the compatibility, system performance and reliability. We also employ the Multi-Logic Device (MLD) mechanism from CXL 2.0 [10], enabling the Fabric Manager to allocate protected regions in a granularity of Logic Device for selective protection.

We have the following contributions in this paper,

- We propose an IC-ECC-enabled CXL Memory eXpander Controller with selective protection. It eliminates the bandwidth consumption of parity updates in the CXL fabric by offloading IC-ECC processing logic to CXL MXC.
- We propose a LRC-based IC-ECC, which enhances the fault tolerance of Channelkill while ensuring the tolerance of numerous random faults. Furthermore, our study conducts the first comprehensive reliability analysis of LRC in the

context of DRAM ECC.

- We evaluated our design using the Champsim [18] and Ramulator2 [19] simulators across various workloads. Our proposed CXL-ECC outperforms state-of-the-art methods, achieving a 60% reduction in bandwidth consumption on average and enhancing DRAM reliability by more than 10^9 , with negligible system performance cost.

II. BACKGROUND AND MOTIVATION

A. CXL Memory eXpander Controller

Compute Express Link (CXL) [10] is a cutting-edge interconnect protocol designed to optimize communication between processors and various devices, such as accelerators and memory expander. Leveraging the physical layer of PCIe 5.0/6.0, CXL facilitates high bandwidth while enabling low-latency, coherent memory access. CXL Memory eXpander Controller (MXC), classified as Type-3 devices, implements CXL.io and CXL.mem protocols to deliver high-performance memory expansion and enable hardware memory pooling. Leading memory manufacturers including Samsung, SK Hynix, and Micron have released CXL MXC products. Marvell also supplies MXC chips [20]. Additionally, internet companies are actively testing MXC performance and modifying systems [13, 21], to support this emerging technology.

B. DIMM Organization and DRAM Faults

1) Dual-Inline-Memory-Module (DIMM) Organization:

DRAM chips have multiple data pins (DQs) that transmit data in parallel, and a chip with N DQs is generally referred to as an xN chip. The DDR5 JEDEC [5] DIMM is equipped with two independent 32-bit sub-channels. In x4 ECC-DIMM, as shown in Fig. 1, each sub-channel is equipped with 10 x4 DRAM chips, where 2 parity chips are used to store the check data of Rank-Level ECC (R-ECC). The internal structure of DRAM chip follows a hierarchical organization, where each consists of multiple independent banks, and each bank can be further divided into rows and columns. DDR5 implements On-Die ECC (OD-ECC), utilizing a (136,128) SEC code for data protection within bank.

2) **DRAM Faults:** Currently, there are two major types of faults in DRAM devices: (i) **Operational Faults** are caused by manufacturing defects and wear-out during usage, and can manifest as cell, row, column, or bank-level faults at a coarser granularity [22]. (ii) **Scaling Faults** appear in the form of random cell faults. As DRAM manufacturing technology scales down, the scaling fault rate increases, reaching as high as 10^{-4} when the process is scaled down to sub-20nm [23]. Each fault introduces random errors, varying from a single bit to 2 symbols in an ECC codeword, and poses significant challenges to the design of high-reliability memory architectures.

C. Error Correcting Code in DRAM

Error Correcting Code (ECC) detects and corrects errors through the addition of redundant information, known as parity, which is algorithmically generated from the protected data. The combination of data and parity forms an ECC codeword.

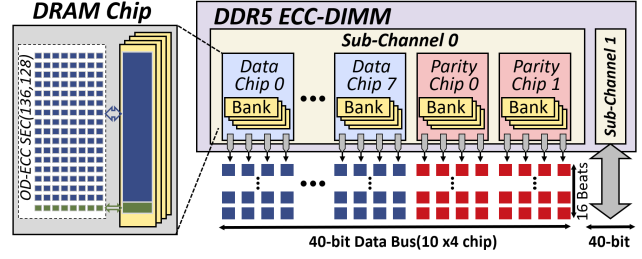


Fig. 1: Organization of x4 DDR5 ECC-DIMM

1) **Distinctions among various ECCs:** Single Error Correction (SEC), Single Error Correction-Double Error Detection (SEC-DED), and Double Error Correction (DEC) are a class of commonly used codes in Rank-Level ECC, which can detect and correct single or multiple bit errors. Another category of codes is represented by Reed-Solomon (RS) codes [2], specifically $RS(n, k)$ codes, which are a type of Maximum Distance Separable (MDS) code. In $RS(n, k)$ codes, n and k represent the total number of symbols and data symbols in the codeword, respectively. When multiple parity chips are available, RS codes can be used to implement Chipkill [4], a fault-tolerant technique that can correct errors even when a single chip completely fails.

2) **Recent Advances in R-ECC :** Researchers have made great efforts on the collaborative design of R-ECC and OD-ECC. XED [24] uses SEC-DED as OD-ECC and replaces the original R-ECC with XOR, enhancing the error detection capability of OD-ECC to improve the error correction capability of R-ECC. DUO [25] encodes the data and R-ECC and OD-ECC parity of a single cache line into a long RS code, achieving Chipkill capability while correcting additional random faults. Unity ECC [26] abandons OD-ECC and selects a coding matrix that satisfies both $RS(10, 8)$ and $BCH(80, 64)$ [27], simultaneously achieving Chipkill and DEC fault tolerance.

However, these techniques have three common issues. (i) The simultaneous modification of both DIMM device and memory controller, and potentially even requiring modifications to the DRAM chip's OD-ECC encoding [24, 25, 28], poses notable compatibility challenges to JEDEC DDR standards [5]. (ii) They require pre-setting the redundancy in the DIMM, which cannot adapt to diverse memory reliability requirements. (iii) To achieve Chipkill, two parity chips are essential, requiring the exclusive use of x4 DRAM chips and affecting DIMM specification design.

D. Challenges with Current Channelkill Technology

To address these aforementioned common issues and further enhance the memory reliability beyond Chipkill, the Cross Channel-Level Fault Tolerance Mechanism (Channelkill) has been proposed and extensively studied, including C-Type and S-Type (defined in § I).

1) **Compute Managed Type (C-Type):** Memory Mirror [6] and IBM-RAIM3 [7] achieve Channelkill fault tolerance by constructing channels as RAID-1/3. RAIM-5 [8] encodes 64B cache lines as RAID-5 stripes, introducing gang-writeback

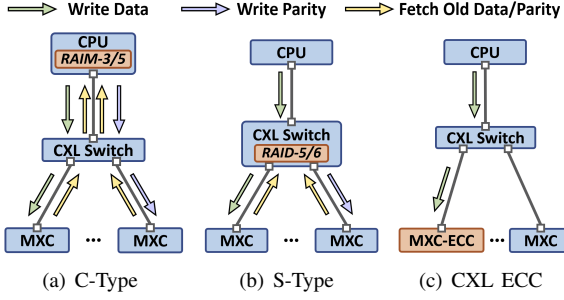


Fig. 2: IO traffic of writeback in three types of Channelkill architecture

and selective protection that enables low-overhead and high-flexibility Channelkill. However, this type of Compute Managed Type (C-Type) Channelkill poses significant challenges to compatibility, because it's nearly impossible for two independent hardware manufacturers to implement a fully compatible complex fault tolerance mechanism.

2) **Switch Managed Type (S-Type)**: This type of approaches address the challenge of memory expander failures within the CXL fabric. SMC [16] proposed integrating replication and RAID-5/6 functionality within the CXL switch, referred to as Switch-RAID. It enables the CXL switch to intercept memory requests from upstream devices and manage parity updates and data recovery, effectively resolving compatibility issues with computing devices.

However, existing Channelkill are still remaining challenges:

- ❶ **Significant overhead incurred by parity update**: The implementation of C/S-Type fault tolerance in CXL fabric, as illustrated in Figures 2(a) and 2(b), introduces a two-step writeback process: reading the original data and parity from cache lines, and writing back their updated versions. These additional operations consume a substantial amount of CXL fabric bandwidth, severely impacting system performance.
- ❷ **XOR-based IC-ECC suffers from poor reliability**: Current IC-ECC widely employs XOR Code for correcting single cache line errors, which fails to correct errors when multiple channels encounter moderate faults, such as multiple symbol errors in RS codes. The root cause is that XOR does not account for the correlation between error distribution in cache lines and DRAM device failure patterns, a detailed explanation will be presented in § III-A.

3) **MXCs serve as an optimal position for IC-ECC logic**: The latest CXL MXCs are equipped with 16 CXL lanes and 4 DDR5 6400MHz channels [20]. Although the total read/write bandwidth of the CXL fabric is limited to 80GB/s, the device's internal bandwidth exceeds 200GB/s. Combining this with the analysis of the above challenges, this motivates us to integrate the IC-ECC processing logic into the memory expander, so that the additional memory requests generated by IC-ECC parity updates can be limited within the expander, effectively alleviating the bandwidth pressure on the CXL fabric. Therefore, we propose the CXL-Memory-Expander Controller ECC (CXL-ECC).

4) **LRC exhibits potential as IC-ECC**: Locally Recoverable Code (LRC) [17] is a variant of RS codes, denoted as $LRC(n, k, r, p)$, where n denotes total code length, k denotes

data length, r denotes global parity length, and p denotes local parity length. The data is divided into $\frac{k}{p}$ local parity groups, each forming an $RS(\frac{k}{p} + p, \frac{k}{p})$ code with local parity. The r global parity blocks are computed from the total k data blocks. LRC exhibits excellent ① **Local Recovery**, ② **Incremental Parity Update** and ③ **High Global Reliability** properties that satisfy the requirements of IC-ECC. LRC is widely used as an erasure code in cloud storage systems [29], but its application in memory systems for ECC has not been explored yet to the best of our acknowledgment.

III. DESIGN OF PROPOSED CXL-ECC

CXL-ECC consists of two main components: (i) LRC-based Inter-Channel ECC (IC-ECC) algorithm design. (ii) IC-ECC-enabled CXL-MXC controller design.

A. Design of LRC-based IC-ECC Algorithm

Drawing upon the analysis presented in § II-C, we propose LRC-based IC-ECC as a solution to address the challenges outlined in § II-D. LRC maintains Single-Symbol-Correction (SSC) capability within each data channel, eliminating the need to read all cache lines that constitute IC-ECC during error detection and correction. By incorporating local parity into the global decoding process, the system achieves robust correction capabilities for numerous random symbol faults.

1) **Code Construction**: Optimal LRC codes [30] conventionally enforce strict constraints on codeword parameters, thereby restricting the number of global parities to not exceed the data symbols within a local group. Therefore, we adopt Pyramid Code [31] for IC-ECC implementation, as it provides equivalent data symbol fault tolerance. Leveraging RS codes constructed with a Vandermonde matrix over $GF(256)$, we implement the code through the following steps:

Step I: We set the number of data channels to be c , then the corresponding Vandermonde matrix for $RS(8 \times c + 12, 8 \times c)$ is calculated as shown in Equation 1:

$$G_V = \begin{bmatrix} (2^0)^0 & (2^0)^1 & \dots & (2^0)^{8 \times c - 1} \\ (2^1)^0 & (2^1)^1 & \dots & (2^1)^{8 \times c - 1} \\ \vdots & \vdots & \ddots & \vdots \\ (2^{11})^0 & (2^{11})^1 & \dots & (2^{11})^{8 \times c - 1} \end{bmatrix} \quad (1)$$

Step II: Let $i = 0, 1, \dots, c - 1$. We align and partition the first two rows of G_V into segments of 8 columns each to obtain the encoding matrix A_i within the local parity group. As demonstrated in Equation 2, forms a generator matrix for a local parity group constructing an $RS(10, 8)$ codeword, thus satisfying the local recovery requirements of LRC.

$$A_i = \begin{bmatrix} 1^{8i} & 1^{8i+1} & \dots & 1^{8i+7} \\ 2^{8i} & 2^{8i+1} & \dots & 2^{8i+7} \end{bmatrix} \quad (2)$$

Step III: We define the global coefficients of the LRC generator matrix shown in Equation 3:

$$G_G = \begin{bmatrix} (2^2)^0 & (2^2)^1 & \dots & (2^2)^{8 \times c - 1} \\ (2^3)^0 & (2^3)^1 & \dots & (2^3)^{8 \times c - 1} \\ \vdots & \vdots & \ddots & \vdots \\ (2^{11})^0 & (2^{11})^1 & \dots & (2^{11})^{8 \times c - 1} \end{bmatrix} \quad (3)$$

Step IV: We define the generator matrix of the LRC as Equation 4, the construction of our proposed LRC-based IC-ECC is completed:

$$G_{LRC} = \begin{bmatrix} A_0 & & I & & \\ & A_1 & & & \\ & & \dots & & \\ & & & A_{c-1} & \\ & & & & G_G \end{bmatrix} \quad (4)$$

2) **Data Encoding and Organization:** Identical to the § III-A1, we assume a total of c data cache lines, encoded into $c + 1$ cache lines using LRC. Fig. 3 illustrates the data layout of cache lines in the IC-ECC structure: the 80 bits received from two consecutive Beats are first aligned, then divided into 10 symbols according to the 4-DQ width to form an IC-ECC coding unit. Let C_{ij} denote the data transmitted in the j -th coding unit within the i -th cache line, where $i \in \{0, 1, \dots, c\}$ and $j \in \{0, 1, \dots, 7\}$. These units are encoded into an LRC($10 \times (c + 1), 8 \times c, 10, 2$) code, with $C_{c,j}$ containing the global parities and each remaining $C_{i,j}$ constituting a local parity group. Cache lines are arranged in a RAID5 left-asymmetric layout [9] to minimize the I/O bottleneck caused by parity update operations.

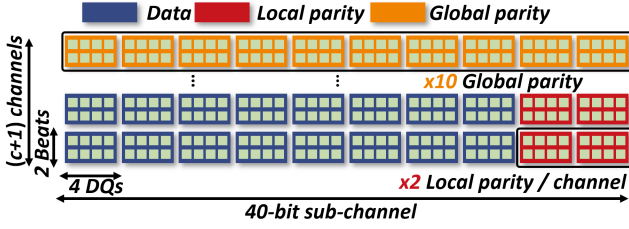


Fig. 3: an example of IC-ECC data layout

3) **Decoding Process and Correction Capability:** IC-ECC employs a two-level error correction scheme: (a) **Intra-channel local decoding:** R-ECC possesses SSC capability, which can achieve Chipkill on x4 DIMMs and Half-Chipkill on x8 DIMMs respectively. When R-ECC correction fails, an Uncorrectable Error (UE) is reported, triggering global decoding. (b) **Inter-channel global decoding:** All data and parity bits across channels are combined into a global RS($c \times 8 + 12, c \times 8$). This allows for either the recovery of a single faulty channel through erasure decoding or the correction of up to 6 random symbol errors using the RS Decoder.

TABLE II: Configurations and error correction capabilities of IC-ECCs

		w/o	XOR	RS	LRC
IC-ECC		-	XOR(5, 4)	RS(40, 32)	RS(44, 32)
Global Parities		-	64B $\times$ 1	1B $\times$ 8	1B $\times$ 12
Correctable	min	1	3	4	6
symbol errors	max	$c + 1$	$c + 10$	$c + 10$	$c + 10$
Chipkill w/ x4 DIMM		1	3	4	6
Chipkill w/ x8 DIMM		0	1	2	3

4) **Compare performance of IC-ECCs within Channelkill:** XOR-based IC-ECC generates parity by XORing data chips within each channel across 64B cache lines. RS-based IC-ECC, similar to LRC-based IC-ECC, uses data chips in the parity channel to store parity symbols that protect data chips in other channels. Table II presents the configurations and

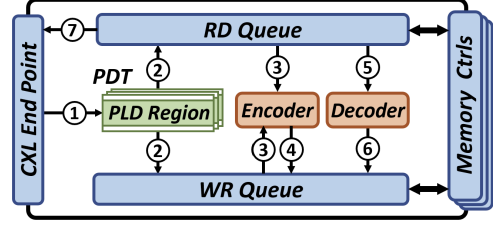


Fig. 4: Architecture of the CXL-ECC controller

error correction capabilities of the three IC-ECCs with 4 data channels ($c = 4$). Due to its coarse encoding granularity, XOR-based IC-ECC exhibits limited tolerance for fine-grained errors, failing to correct dual-symbol errors occurring simultaneously in two channels. While RS-based IC-ECC offers superior fine-grained error correction compared to XOR, its local parity does not contribute to global decoding, limiting its correction capability to 4 arbitrary symbol errors. By contrast, LRC-based IC-ECC integrates local and global redundancy to form 12 parity symbols.

B. Design of CXL-ECC Controller System Architecture

The system architecture of CXL-ECC implements selective protection management, parity update procedures, and data recovery mechanisms.

1) Selective protection with Protected LD Table (PDT):

Multiple Logic Device (MLD) characteristic enables each Type-3 device to be logically partitioned into Logic Devices (LDs) and independently assigned to computing devices. Memory space is protected by IC-ECC at the LD granularity, with all protected LDs being recorded in the Protected LD Table (PDT). During high-reliability memory allocation by the Fabric Manager (FM), a sequential process is executed wherein: (1) the FM initiates an allocation request to the MXC, (2) the MXC performs physical space allocation within its internal DDR channels and registers the allocation in the PDT, and (3) the resultant protected LD is subsequently exported and assigned through the FM.

2) Parity update and data recovery via En-/De-coder:

The processes for parity updates and data recovery of IC-ECC are executed by the En-/De-coder modules. (i) **Request Serve** begins with ① a filtering in the PDT. If an operation falls within the PDT, ② then while the request is placed in the RQ/WQ, the protected flag needs to be set in the corresponding entry. Read requests for previously written data and parity required for the incremental update are inserted into the RQ. ③ Upon completion of all read requests for a writeback operation, both new and previous data, along with the old parity, are passed to the Encoder. ④ A write request is issued for the encoded result. (ii) **Data Recovery** is triggered upon the detection of a R-ECC UE¹ in an entry with protected flag set. Subsequently, read requests associated with data recovery are enqueued in the RQ. ⑤ When all data required for the recovery operation has been retrieved, the Decoder fetches the data and performs

¹To enable the IC-ECC En-/De-coder to access data from the parity chips, we relocate the R-ECC encoding and decoding operations from the memory controller to the MXC.

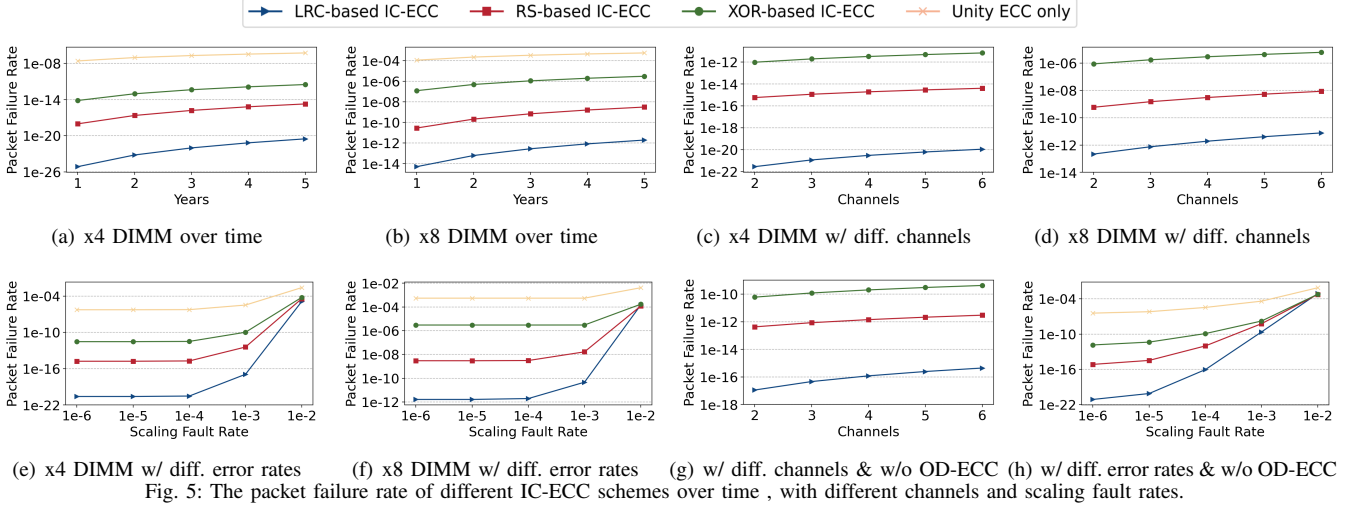


Fig. 5: The packet failure rate of different IC-ECC schemes over time, with different channels and scaling fault rates.

the decoding. ⑥ Following this, write requests for the decoded results are queued in the WQ. If the R-ECC UE is not in the PDT or the Decoder fails to decode, an UE is reported to the CXL End Point.

3) Scheduler optimization and conflicts management:

The First-Ready First-Come-First-Serve (FR-FCFS) scheduling policy is modified to assign higher priority to CPU load requests when multiple read requests have equivalent priority levels. This optimization minimizes the impact on load latency caused by the data reads required for IC-ECC parity updates. To maintain data consistency during simultaneous parity updates and data recovery operations on the same IC-ECC codeword, operations are serialized, prioritizing data recovery to avoid deadlocks.

TABLE III: Operational fault rates[22]

Mode	Transient	Permanent
Single bit	14.2	18.6
Single word	1.4	0.3
Single column	1.4	5.6
Single row	0.2	8.2
Single bank	0.8	10.0

IV. EVALUATION METHOD

A. Reliability Simulation Setup

To evaluate the reliability of LRC-based IC-ECC, we compare it with schemes using only Unity ECC (SOTA Rank-level ECC) as R-ECC, Xor-based IC-ECC (incorporated by both RAIM-5 and Switch-RAID) and RS-based IC-ECC with Unity ECC. The experiments employ 2-6 data channels plus one parity channel to construct IC-ECC with DDR5 x8 and x4 ECC-DIMM². During fault injection process, random cell faults are inserted with a probability between 10^{-6} and 10^{-2} to simulate Scaling Faults, then Operational Faults are injected using the failure rates shown in Table. III [22] to simulate wear-out after 1-5 years of device operation. As the probability of UE occurring in LRC-based IC-ECC is extremely low (in

²In the default configuration, the system employs 4 data channels, maintains a scaling fault rate of 10^{-4} , operates over a 5-year period, and implements OD-ECC protection for DRAM chips.

some cases less than 10^{-20}), making it difficult to simulate the failure rate using Monte Carlo methods. We theoretically calculate the packet failure rate corresponding to 5×80 bits data constituting an IC-ECC.

B. Performance Simulation Setup

TABLE IV: The system simulation configuration

Processor	
Cores	16 cores 4GHz, 4-wide OoO
L1I	32KB, 64B line, 8 way, 4 cycles, 8 MSHRs
L1D	48KB, 64B line, 12 way, 5 cycles, 16 MSHRs
L2C	512KB, 64B line, 8 way, 10 cycles, 32 MSHRs
LLC	32MB, 64B line, 16 way, 20 cycles, 1024 MSHRs
CXL Fabric [13]	
Latency	CPU Port: 25ns, Switch: 130ns, Memory Port: 40ns
Bandwidth	x8 CXL (PCIe5.0) lanes, 32 GB/s (bi-directional)
Memory eXpander Controller	
Queue Depth	MXC: 1024, DDR controller: 32
DDR Spec.	1 rank, DDR5 [5] 6400MHz, 32Gb x8 chip, 8 bank groups, 4 banks per bank group
Channels	5 channel (4-data + 1-parity)

To evaluate the system-level performance of the CXL-ECC architecture, we integrate the Champsim [18] CPU simulator with the Ramulator2³ [19] Memory simulator, implementing the CXL protocol in Ramulator2. We also implement RAIM-5, RAID5-enabled Switch (Switch-RAID), and CXL-ECC. We evaluate 145 traces sampled from 41 workloads from SPEC CPU2006 [32], SPEC CPU2017 [33], PARSEC 2.1 [34], and Ligr [35]. During simulation execution, we run the same trace on all CPUs, measuring the execution time and CXL fabric traffic for around 100M instructions (after 10M warmup). Table IV summarizes all configurations used in the system simulation.

V. EVALUATION RESULTS

A. Impact on Reliability

(a) CXL-ECC substantially enhances DRAM reliability.

Fig. 5(a) and Fig. 5(b) show the packet error rate over time

³Ramulator2 only support single sub-channel per DDR5 channel, we halve the CXL fabric bandwidth to simulate realistic DDR5 performance.

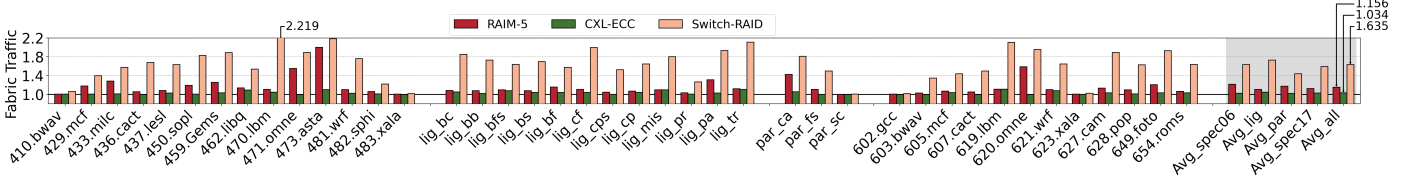


Fig. 6: Normalized fabric traffic of RAIM-5, Switch-RAID and CXL-ECC on each benchmark suite and the overall average

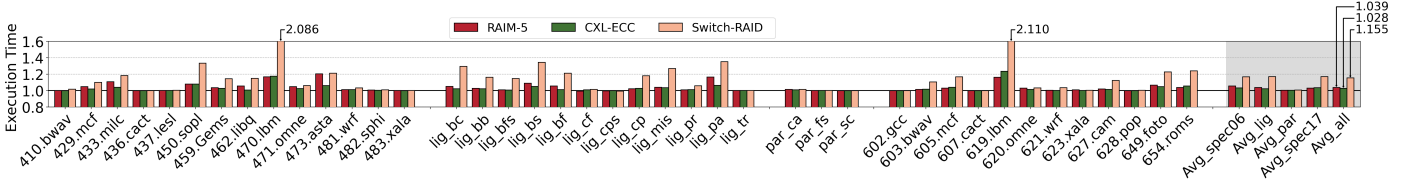


Fig. 7: Normalized execution time of RAIM-5, Switch-RAID and CXL-ECC on each benchmark suite and the overall average

for x4 and x8 DDR5 ECC-DIMM using different IC-ECC mechanisms. Equipped with x4 DIMM, CXL-ECC can achieve a packet failure rate below 3.1×10^{-21} while providing tolerance for complete failures of individual DIMMs, enabling highly reliable memory expansion for computing devices in CXL fabric. In comparison to the XOR-based ECC employed in existing Channelkill mechanisms, LRC-based IC-ECC yields reliability improvements of 6.3×10^5 and 10^9 in x8 and x4 DIMMs, respectively. Furthermore, LRC exhibits superior global error correction capabilities compared to refined RS codes, resulting in reliability improvements of 1.6×10^3 and 1.5×10^6 in x8 and x4 DIMMs, respectively.

Although x8 DIMMs inherently lack Chipkill capabilities, the use of LRC-based IC-ECC enables them to tolerate up to 3 random chip failures, thereby demonstrating robust resilience against both scaling and operational faults. Furthermore, the packet failure rate of x8 DIMMs employing LRC-based IC-ECC remains below 2×10^{-12} , which is lower than the 3×10^{-12} rate of x4 DIMMs protected by XOR-based IC-ECC.

(b) CXL-ECC demonstrates robust reliability across numerous channels and high failure rates. Fig. 5(c), 5(d), 5(e), and 5(f) illustrate the reliabilities of various IC-ECC mechanisms under different channel configurations and scaling error rates. Experimental results demonstrate that LRC-based IC-ECC-protected x4 DIMMs maintain an exceptionally low failure rate of 1.2×10^{-17} even when the scaling fault rate reaches 10^{-3} . Even with up to 6 data channels in x8 DIMMs, a minimal error rate of 7.8×10^{-12} can be maintained.

(c) CXL-ECC sustains exceptional reliability even in the absence of OD-ECC. Fig. 5(g) and 5(h) show the relationship between failure rates and varying parameters (channel count and error rate) for x4 DDR5 DIMMs without OD-ECC. Compared to OD-ECC-enabled configurations, all IC-ECC schemes show diminished error correction capabilities. Among these, LRC-based IC-ECC exhibits the most severe reliability degradation under increased scaling errors. This phenomenon can be attributed to the SSC-only mechanism of R-ECC within data channels, exhibiting inferior random error tolerance compared to the DEC-enabled Unity ECC deployed in other IC-ECCs. At an error rate of 10^{-3} , LRC maintains superior

reliability despite these limitations, showing 29x and 98x improvements over RS-based and XOR-based implementations, respectively.

B. Impact on Performance

(a) CXL-ECC significantly reduces fabric bandwidth overhead. As shown in Fig. 6, the average additional bandwidth consumed by CXL-ECC across all workloads is 3.4%, which is significantly lower than the 15.6% and 63.5% of RAIM-5 and Switch-RAID, respectively. This is because all extra memory requests related to parity updates are confined within the MXC. For write-intensive applications such as GemsFDTD, 1bm, and BFS, CXL-ECC can reduce the additional CXL fabric bandwidth overhead by up to 114% compared to S-type Switch-RAID technology.

It is worth noting that some applications, such as omnetpp, have a large additional bandwidth overhead but do not affect system performance. This is because they are not memory-intensive applications, and the additional memory requests do not put pressure on the CXL fabric.

(b) CXL-ECC has negligible impact on overall system performance. Fig. 7 shows the normalized execution time of CXL-ECC, Switch-RAID, and RAIM compared to the baseline system without IC-ECC. The S-Type Switch-RAID mechanism causes an execution time increase of over 13.4%, significantly impacting system performance. This is because a large amount of fabric bandwidth is used for parity updates. The workloads with severe performance degradation in Fig. 7 generally generate high fabric traffic in Fig. 6.

CXL-ECC and RAIM-5 exhibit marginal performance degradation of 2.8% and 3.9% respectively, significantly outperforming Switch-RAID with its substantial 15.5% overhead. Furthermore, our approach surpasses traditional C-type solutions like RAIM-5 while eliminating the need for extensive CPU modifications. Experimental results indicate that CXL-ECC delivers superior reliability and system-level performance while maintaining architectural compatibility.

VI. CONCLUSION

This paper presents on-CXL-Memory-Expander ECC (CXL-ECC) which integrates LRC-based IC-ECC in the CXL

memory controller. This approach effectively leverages the robust global error correction capabilities of LRC and mitigates the bandwidth demands imposed by parity updates on the fabric, thereby enabling the realization of highly reliable memory, which enhances a UE rate of lower than 3.1×10^{-21} , with minimal system performance overhead of 2.8%.

VII. ACKNOWLEDGEMENTS

We thank all the anonymous reviewers for their valuable comments. This work was supported in part by the National Key R&D Program of China (Grant No. 2023YFB4502900).

REFERENCES

- [1] I. Corporation. Intel mca+mfhp helps jd stable and efficient cloud services.
- [2] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the society for industrial and applied mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
- [3] R. W. Hamming, "Error detecting and error correcting codes," *The Bell system technical journal*, vol. 29, no. 2, pp. 147–160, 1950.
- [4] AMD. Bios and kernel developers guide (bkdg) for amd family 15h models 00h-0fh processors.
- [5] M. JEDEC. ddr5 sdram standard, jesd79-5c.01.
- [6] I. Corporation. Address range partial memory mirroring.
- [7] P. J. Meaney *et al.*, "Ibm zenterprise redundant array of independent memory subsystem," *IBM Journal of Research and Development*, vol. 56, no. 1.2, pp. 4–1, 2012.
- [8] R. Zheng and M. C. Huang, "Redundant memory array architecture for efficient selective protection," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 214–227, 2017.
- [9] P. M. Chen *et al.*, "Raid: High-performance, reliable secondary storage," *ACM Computing Surveys (CSUR)*, vol. 26, no. 2, pp. 145–185, 1994.
- [10] C. Consortium. Compute express link (cxl) specification, revision 3.1. [Online]. Available: <https://computeexpresslink.org/cxl-specification>.
- [11] S. Eletronics. Samsung cxl solutions – cmm-h.
- [12] S. N. I. Association. Persistent memory in cxl. [Online]. Available: <https://www.snia.org/sites/default/files/PM-Summit/2021/sni-a-pm-cs-summit-Rudoff-PM-in-CXL-2021.pdf>.
- [13] H. Li *et al.*, "Pond: Cxl-based memory pooling systems for cloud platforms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 574–587.
- [14] J. Liu *et al.*, "Exploring and evaluating real-world cxl: Use cases and system adoption," *arXiv preprint arXiv:2405.14209*, 2024.
- [15] Y. Tang *et al.*, "Exploring performance and cost optimization with asic-based cxl memory," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 818–833.
- [16] D. D. Sharma, "Novel composable and scaleout architectures using compute express link," *IEEE Micro*, vol. 43, no. 2, pp. 9–19, 2023.
- [17] I. Tamo and A. Barg, "A family of optimal locally recoverable codes," *IEEE Transactions on Information Theory*, vol. 60, no. 8, pp. 4661–4676, 2014.
- [18] N. Guber *et al.*, "The championship simulator: Architectural simulation for education and competition," *arXiv preprint arXiv:2210.14324*, 2022.
- [19] H. Luo *et al.*, "Ramulator 2.0: A modern, modular, and extensible dram simulator," *IEEE Computer Architecture Letters*, 2023.
- [20] M. Inc. Structera x 2504 memory-expansion controller.
- [21] Y. Sun *et al.*, "Demystifying cxl memory with genuine cxl-ready systems and devices," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 105–121.
- [22] V. Sridharan and D. Liberty, "A study of dram failures in the field," in *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 2012, pp. 1–11.
- [23] P. J. Nair *et al.*, "Archshield: Architectural framework for assisting dram scaling by tolerating high error rates," *ACM SIGARCH Computer Architecture News*, vol. 41, no. 3, pp. 72–83, 2013.
- [24] P. J. Nair *et al.*, "Xed: Exposing on-die error detection information for strong memory reliability," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 341–353, 2016.
- [25] S.-L. Gong *et al.*, "Duo: Exposing on-chip redundancy to rank-level ecc for high reliability," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 683–695.
- [26] D. Kim *et al.*, "Unity ecc: Unified memory protection against bit and chip errors," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–16.
- [27] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Information and control*, vol. 3, no. 1, pp. 68–79, 1960.
- [28] S. Jeong *et al.*, "Pair: Pin-aligned in-dram ecc architecture using expandability of reed-solomon code," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [29] G. Yang *et al.*, "Xhr-code: An efficient wide stripe erasure code to reduce cross-rack overhead in cloud storage systems," in *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2022, pp. 273–283.
- [30] G. Zhang, "A new construction of optimal (r, δ) locally recoverable codes," *IEEE Communications Letters*, vol. 24, no. 9, pp. 1852–1856, 2020.
- [31] C. Huang *et al.*, "Pyramid codes: Flexible schemes to trade space for access efficiency in reliable data storage systems," *ACM Transactions on Storage (TOS)*, vol. 9, no. 1, pp. 1–28, 2013.
- [32] Spec cpu 2006. [Online]. Available: <https://www.spec.org/cpu2006/>.
- [33] Spec cpu 2017. [Online]. Available: <https://www.spec.org/cpu2017/>.
- [34] Parsec. [Online]. Available: <https://parsec.cs.princeton.edu/>.
- [35] J. Shun and G. E. Blelloch, "Ligra: a lightweight graph processing framework for shared memory," in *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, 2013, pp. 135–146.